

SQL-Agents: Sistema Multi-Agente para la Generación de Consultas SQL desde Lenguaje Natural

Félix David Córdova García
fdcordova@javeriana.edu.co

Edison Leonardo Neira Espitia
edison-neira@javeriana.edu.co

Pontificia Universidad Javeriana – Bogotá D.C.

Abstract. This paper presents **SQL-Agents**, a multi-agent NL (Natural Language) to SQL system that makes relational databases accessible to non-technical users. Six specialized agents —AR (Refiner), APS (Schema Matcher), AG (SQL Generator), AV (Validator), AE (Explainer) and AS (Sustainer)— follow the **Agent Skills** [36] pattern, orchestrated by a central agent with intent classification and short/long-term memory. The **Model Context Protocol (MCP)** [37] externalizes domain knowledge, decoupling it from the LLM provider. The system is evaluated over **five phases** using the Spider 1.0 **concert_singer** database over MySQL and PostgreSQL: **Phase 0** selects candidate models and evaluation frameworks via an exhaustive model $\times$ temperature grid on the Refiner agent; **Phase 1** independently optimizes each agent with Optuna TPE [27, 48] using role-specific metrics; **Phase 2** validates the paraphrase robustness of individual agents across three linguistic reformulations per intent; **Phase 3** evaluates and optimizes the end-to-end orchestrator over ten pipeline scenarios using **ToolCorrectnessMetric**; **Phase 4** assesses orchestrator robustness under thirty paraphrases, yielding a normalized pipeline confidence index of **82.9 %** for MySQL and **83.2 %** for PostgreSQL (combined score over a theoretical maximum of 3.0), suggesting stable end-to-end pipeline behavior and semantic consistency across the evaluated scenarios.

Keywords: Text-to-SQL, multi-agent system, Agent Skills, Model Context Protocol, conversational context, orchestrator, vector similarity, proactive validation, explainability, LLM, ToolCorrectnessMetric, WACS, Faithfulness (F), Groundedness (G), Brier Score (B), ECE (E), Optuna TPE.

1. Introducción

El uso de bases de datos relacionales está principalmente asociado a personas que conocen el lenguaje SQL [1]. NL-to-SQL propone convertir preguntas en lenguaje natural a consultas SQL [8], pero tres limitaciones persisten: validación *post-hoc* [3], ambigüedad semántica [4] y ausencia de gestión de contexto conversacional [15].

Frente a enfoques monolíticos, descomponer el pipeline en **agentes especializados** reduce la alucinación: cada agente opera sobre un contexto acotado con herramientas propias, lo que limita la propagación de errores y permite corregirlos de forma local antes de que afecten las etapas siguientes [10]. Los experimentos sugieren que distintos modelos son óptimos para cada rol, aportando evidencia a favor de la arquitectura multi-agente frente a un único LLM centralizado.

Este trabajo propone **SQL-Agents**, un sistema multi-agente que aborda las tres limitaciones. El **Orquestador** gestiona el contexto conversacio-

nal antes de invocar el pipeline: detecta cambios de dominio, resuelve referencias anafóricas y consulta memorias a corto plazo (caché RAM) y largo plazo (ChromaDB). Si las memorias producen un acierto, retorna el resultado sin invocar el pipeline. En caso contrario, ejecuta los seis agentes especializados: **AR** (refinador), **APS** (proximidad semántica), **AG** (generador SQL), **AV** (validador iterativo), **AE** (explicador) y **AS** (sustentador bajo demanda).

El protocolo experimental se organiza en **cinco fases**. La **Fase 0** compara marcos de evaluación sobre el AR con DeepEval como motor compartido de métricas; ejecuta además una grilla exhaustiva de 30 combinaciones (6 modelos $\times$ 5 temperaturas), determinando las configuraciones candidatas para las fases siguientes; selecciona Optuna como herramienta de optimización. La **Fase 1** optimiza independientemente cada agente mediante Optuna TPE [27, 48] sobre el espacio modelo $\times$ temperatura, evaluando con **10 preguntas** de prueba y métricas adaptadas al rol de

cada agente. La **Fase 2** evalúa la robustez ante paráfrasis: fija las configuraciones ganadoras de Fase 1 y reformula cada pregunta en 3 variantes lingüísticas, midiendo cómo varía la calidad de respuesta de cada agente. La **Fase 3** evalúa el orquestador ejecutando el pipeline completo sobre **10 escenarios** que cubren todas las rutas posibles del sistema, usando `ToolCorrectnessMetric` para verificar la activación correcta de agentes; Optuna busca simultáneamente el mejor modelo y temperatura para el orquestador. La **Fase 4** evalúa la robustez del orquestador ante **30 paráfrasis** (3 reformulaciones por escenario).

Limitaciones del alcance. El sistema opera únicamente sobre bases de datos relacionales (OLTP); fuentes NoSQL, documentales o analíticas (data warehouse) quedan fuera del alcance. Este trabajo no incluye la ejecución efectiva de las consultas SQL contra la base de datos ni el tratamiento de tipos de datos complejos como arreglos (ARRAY), tipos JSON anidados o datos geoespaciales. El sistema genera, valida y explica la consulta en lenguaje natural, pero delega la ejecución y el manejo de resultados estructurados complejos a la capa de aplicación que lo integre. Debido al alto consumo de tokens se realizó un conjunto de evaluación de 10 casos por agente, que permitieron una exploración inicial del comportamiento del sistema.

2. Estado del Arte

La investigación en NL-to-SQL ha evolucionado notablemente con la integración de LLMs y arquitecturas basadas en agentes, pero los avances siguen siendo fragmentados: algunos sistemas fortalecen la generación, otros la recuperación contextual, y pocos abordan la validación, sin que —entre los trabajos revisados— alguno integre los tres ejes junto con gestión de contexto conversacional dentro de una arquitectura unificada.

Ojuri et al. [1] integran LLMs con agentes mediante *ReAct*, demostrando que el *fine-tuning* supera al aprendizaje en contexto, pero sin verificación ni explicaciones integradas. Tai et al. [4] aplican razonamiento encadenado (*chain-of-thought*)

mejorando la generación, pero sin validación ni retroalimentación.

En recuperación contextual, Zhao et al. [2] proponen *Chat2Data* con bases vectoriales para reducir alucinaciones. Vichev y Marchev [5] evalúan estrategias de *embeddings* para recuperación de esquema. Ambos evidencian el valor de la recuperación semántica, pero carecen de validación y no contemplan el historial del usuario.

Desde la generación multi-agente, Wang et al. [19] proponen MAC-SQL, descomponiendo la generación SQL en sub-tareas especializadas coordinadas por un orquestador; sin embargo, no contemplan validación proactiva ni memoria conversacional. Wang et al. [3] proponen un agente que usa herramientas para inspeccionar y refinar consultas SQL, detectando discrepancias entre el SQL generado y los datos reales, pero con verificación *post-hoc* sin retroalimentar la generación.

Desde la perspectiva de *benchmarks*, los trabajos de referencia miden **EX** (*Execution Accuracy*: porcentaje de consultas cuyo resultado de ejecución coincide exactamente con el del SQL *gold*). Yu et al. [32] introducen Spider 1.0 (10.181 pares pregunta-SQL, 200 bases de datos), estándar entre 2018 y 2023; sobre él, DIN-SQL [33] alcanza 82.8 % EX y DAIL-SQL [34] 86.6 % EX, ambos con *fine-tuning* o *few-shot* sobre el propio *split* de entrenamiento. BIRD [35] (2023) introduce 12.751 pares sobre 95 bases de datos reales; GPT-4 obtiene 54.9 % EX recibiendo el DDL completo en cada *prompt* más un campo *evidence* anotado manualmente. Spider 1.0 se utiliza en este trabajo únicamente como fuente de esquema y casos de estudio, no como benchmark competitivo. Estos sistemas no son directamente comparables con SQL-Agents porque (1) reciben el DDL completo como *input* (inviable en producción), (2) dependen de datos de entrenamiento o anotaciones manuales por dominio, y (3) carecen de contexto conversacional, memoria y validación proactiva. SQL-Agents no reporta EX porque no ejecuta consultas contra la base de datos durante la evaluación; en su lugar mide similitud textual canonicalizada del SQL generado (`rouge_1_sql`) y calidad de la explicación (Faithfulness, Groundedness).

La Tabla 1 sugiere que, entre los trabajos revisados durante esta investigación, no se identificaron trabajos que cubran simultáneamente proximidad vectorial, validación iterativa, refinamiento semántico, gestión de contexto conversacional, explicabilidad y memoria conversacional.

Tabla 1: Comparación de trabajos relacionados.

Trabajo	Prox.	Valid.	Refin.	GCC	Gener.	Sust.	Mem.	Skills	MCP	EscProd.
Ojuri et al. [1]	No	No	No	No	Sí	No	No	No	No	No
Tai et al. [4]	No	No	No	No	Sí	No	No	No	No	No
Zhao et al. [2]	Sí	No	No	No	Sí	No	No	No	No	No
Vichev & M. [5]	Sí	No	No	No	Sí	No	No	No	No	No
MAC-SQL [19]	No	No	No	No	Sí	No	No	No	No	No
Wang et al. [3]	No	Parc.	No	No	Sí	No	No	No	No	No
DIN-SQL [33]	No	No	No	No	Sí	No	No	No	No	No
DAIL-SQL [34]	No	No	No	No	Sí	No	No	No	No	No
SQL-Agents	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí

Prox.: Proximidad Vectorial. **Valid.:** Validación proactiva. **Refin.:** Refinamiento semántico. **GCC:** Gestión de Contexto. **Sust.:** Sustentador. **Mem.:** Memoria inter-sesión. **MCP:** Model Context Protocol. **EscProd.:** escalable a producción sin DDL completo ni entrenamiento por dominio.

Desde la perspectiva arquitectónica, el patrón **Agent Skills** [36] —propuesto por Anthropic para estructurar agentes LLM como habilidades modulares e invocables— no se encontró evidencia en la literatura revisada de aplicaciones explícitas de este patrón a sistemas NL-to-SQL. Tampoco se identificaron trabajos revisados que reporten explícitamente el uso del **Model Context Protocol (MCP)** [37] para exponer herramientas del esquema de base de datos a los agentes. Los trabajos existentes adoptan arquitecturas monolíticas o pipelines secuenciales fijos, sin separar las responsabilidades en Skills independientes ni utilizar protocolos estandarizados de acceso al contexto. SQL-Agents fue diseñado para el uso de ambos patrones, permitiendo que cada agente sea evaluado, reemplazado o escalado de forma aislada sin afectar al resto del sistema.

3. Marco Teórico

3.1 NL-to-SQL

La tarea NL-to-SQL consiste en transformar una pregunta en lenguaje natural en una consulta SQL ejecutable sobre un esquema relacional dado. Formalmente, dado un enunciado q y un esquema S , el objetivo es producir $y = f(q, S)$ [8]. Los principales desafíos incluyen la resolución de ambigüedades léxicas, la alineación entre términos del usuario y nombres de columnas, y la generación correcta de *joins* en esquemas complejos [9]. El alcance del presente trabajo se restringe a bases de datos relacionales OLTP con propiedades ACID y esquemas en estrella o copo de nieve, excluyendo fuentes NoSQL o documentales.

3.2 Modelos de Lenguaje Grandes (LLMs)

Los LLMs son redes *transformer* entrenadas sobre corpus masivos, capaces de realizar tareas

de comprensión y generación de lenguaje natural sin entrenamiento específico por tarea [7]. En NL-to-SQL, han demostrado capacidad para generar consultas complejas cuando se les provee contexto del esquema mediante *prompting* estructurado [33]. La **temperatura** T controla la distribución de probabilidad sobre los tokens generados: $T \approx 0$ produce salidas deterministas, mientras que $T \approx 1$ incrementa la diversidad léxica pero puede reducir la precisión [7].

3.3 Sistemas Multi-Agente

Un sistema multi-agente (SMA) es un entorno computacional compuesto por agentes autónomos que perciben su entorno, razonan y ejecutan acciones para alcanzar objetivos colectivos [10]. En el ámbito de los LLMs, los agentes son instancias configuradas con roles, herramientas y restricciones específicas que colaboran mediante paso de mensajes y estado compartido [11]. La especialización de agentes permite descomponer tareas complejas en subtarefas manejables, mejorando la trazabilidad y la modularidad del sistema [10].

3.4 Búsqueda Semántica por Proximidad Vectorial

La búsqueda semántica por proximidad vectorial es la técnica que permite recuperar información relevante representando textos como vectores numéricos (*embeddings*) y calculando su similitud mediante coseno [12]. A diferencia de la búsqueda por palabras clave, captura el significado de los términos aunque no coincidan literalmente. En el sistema SQL-Agents, el Agente de Proximidad Semántica (APS) utiliza esta técnica para identificar las tablas y columnas del esquema más relevantes para cada pregunta del usuario, reduciendo el contexto enviado al LLM y mejorando la precisión de la consulta generada [5]. ChromaDB es la base de datos vectorial de código abierto empleada para almacenar, indexar y consultar estos *embeddings*

de forma eficiente [13].

3.5 Gestión de Contexto Conversacional

Tres mecanismos interdependientes [15] sustentan el diálogo multi-turno: **Context Window** (ventana de tokens disponibles [7]), **Context Switching** (detección de cambio de dominio para reiniciar el contexto sin perder la memoria) y **Contextual Awareness** (*coreference resolution*: “¿de esos, cuántos son de Perú?” extiende la consulta previa [16]). En SQL-Agents el Orquestador combina los tres con la memoria a corto y largo plazo.

3.6 Memoria Conversacional

La memoria en sistemas LLM se clasifica en dos tipos [15]: la **memoria a corto plazo**, que mantiene el historial de la sesión activa almacenando consultas recientes y sus resultados SQL exitosos para evitar reprocesamiento; y la **memoria a largo plazo**, que persiste entre sesiones información sobre consultas frecuentes y esquemas accedidos. En SQL-Agents, antes de ejecutar el pipeline completo, el Orquestador consulta la memoria a largo plazo: si existe una respuesta similar a la pregunta actual, la reutiliza como contexto inicial sin invocar a los demás agentes [14].

3.7 Change Data Capture (CDC) y Contexto de Esquema

En este trabajo, el módulo CDC es un script Python que se conecta a las bases de datos MySQL y PostgreSQL para extraer el DDL, particiones, índices y estadísticas del esquema, serializándolos en formato JSON [17]. Este JSON, junto con el **diccionario de datos** —un documento que describe el significado semántico de cada tabla, columna y relación—, constituye el contexto que el Agente de Proximidad indexa en ChromaDB.

3.8 Patrón Agent Skills

El patrón **Agent Skills** [36] define cada capacidad de un agente LLM como una *skill*: unidad modular, autocontenida e invocable mediante una interfaz definida. Un orquestador central selecciona dinámicamente qué skill invocar según el estado de la conversación. En SQL-Agents cada agente es una skill registrada en un *Tool Registry*; adicionalmente, cada agente expone sub-skills internas que pueden invocarse de forma independiente. Este diseño permite la evaluación aislada de cada agente —característica central de la metodología experimental— y la sustitución del LLM de cualquier agente sin modificar los demás.

3.9 Model Context Protocol (MCP)

El **Model Context Protocol (MCP)** [37] es un estándar abierto de Anthropic que conecta agentes LLM con servicios externos mediante JSON-RPC sobre stdio o SSE. A diferencia del patrón Agent Skills —donde cada habilidad vive embebida en el agente—, las herramientas MCP residen en un servidor independiente reutilizable por múltiples agentes o clientes externos.

En SQL-Agents, MCP externaliza dos clases de conocimiento:

1. **Contexto de dominio**: descripción narrativa de cada base de datos (propósito, usuarios, preguntas frecuentes) mantenida como documento externo; los agentes la consultan vía MCP sin *hardcodear* reglas.
2. **Operaciones algorítmicas sobre el esquema**: similitud coseno sobre *embeddings* indexados, listado de claves foráneas e identificación de claves primarias —deterministas y sin LLM.

La especialización por base de datos se concentra en archivos externos editables, satisfaciendo el principio de agnosticidad al dominio.

3.10 LangGraph y Orquestación de Agentes

LangGraph modela flujos de agentes LLM como grafos de estado dirigidos, donde cada nodo representa un agente y las aristas determinan transiciones según condiciones definidas [18]. Soporta ciclos de retroalimentación (*feedback loops*), permitiendo que el Agente Validador devuelva el control al Agente Generador para corregir la consulta sin reiniciar el pipeline completo, lo cual es esencial para la validación proactiva en SQL-Agents [18].

3.11 Métricas de Confianza y Evaluación

La evaluación de sistemas NL-to-SQL requiere métricas que midan tanto la calidad de las salidas generadas como la calibración de la confianza reportada por los agentes. En SQL-Agents se emplean seis métricas complementarias, aplicables a cualquier agente del pipeline.

ROUGE-L (*Recall-Oriented Understudy for Gisting Evaluation*) mide la similitud entre la salida generada por un agente y una referencia esperada, utilizando la subsecuencia común más larga (LCS) [6]. A diferencia de otras métricas de similitud textual, ROUGE-L no requiere coincidencias contiguas, lo que permite capturar la preservación del contenido semántico incluso cuando el orden

de las palabras varía. Se calcula como:

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot R_{lcs} \cdot P_{lcs}}{R_{lcs} + \beta^2 \cdot P_{lcs}} \quad (1)$$

donde $R_{lcs} = |LCS(\hat{y}, y)|/|y|$, $P_{lcs} = |LCS(\hat{y}, y)|/|\hat{y}|$ y β pondera precisión frente a cobertura. Un valor cercano a 1 indica que la salida reproduce fielmente el contenido de la referencia.

WACS (*Weighted Agent Confidence Score*) es una métrica propuesta por los autores de este trabajo. Cuantifica la confianza global del sistema como promedio ponderado de la confianza declarada por cada agente:

$$\text{WACS} = \sum_{i \in A} w_i \cdot c_i \quad (2)$$

donde $A = \{\text{AR}, \text{APS}, \text{AG}, \text{AV}\}$, $c_i \in [0, 1]$ es la confianza del agente i , y los pesos son: AR (0,10), APS (0,20), AG (0,30), AV (0,40). Un WACS alto con ROUGE-L bajo señala sobreconfianza, orientando el ajuste de umbrales.

Faithfulness (F). La salida del agente **no contradice ni excede la respuesta esperada** [43]. Para el AR es la versión limpia esperada de la pregunta; para AE/AS, la explicación esperada. Formalmente, sobre el conjunto S de afirmaciones de la salida:

$$F = \frac{|\{s_i \in S : s_i \text{ no contradice lo esperado}\}|}{|S|}$$

Groundedness (G). La salida se ancla en la entrada recibida sin agregar conceptos ausentes [44, 45]:

$$G = \frac{|\{s_i \in S : s_i \text{ se deriva de la entrada}\}|}{|S|}$$

F evalúa contra la **respuesta esperada**; G contra la **entrada recibida**: dos ejes complementarios para detectar fallos de distinta naturaleza [21].

Brier Score (B) mide la calibración probabilística de las confianzas declaradas por los agentes [22]. Cuantifica la diferencia cuadrática media entre la confianza declarada p_i y el resultado binario real $o_i \in \{0, 1\}$:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (p_i - o_i)^2 \quad (3)$$

Un Brier Score (B) cercano a 0 indica que la confianza declarada es un predictor preciso del resultado. Por ejemplo, si un agente declara confianza 0.9 y acierta, su Brier individual es $(0.9 - 1.0)^2 = 0.01$; si declara 0.9 pero falla, es $(0.9 - 0.0)^2 = 0.81$.

Expected Calibration Error (E) complementa el Brier Score agrupando las predicciones en M intervalos (*bins*) por nivel de confianza y midiendo la diferencia entre la confianza media y la precisión real en cada intervalo [23]:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (4)$$

Un ECE (E) cercano a 0 indica que un agente que dice estar “80 % seguro” acierta efectivamente en el 80 % de los casos. A diferencia del Brier Score que da un valor global, ECE identifica en qué rangos de confianza el agente está mal calibrado.

4. Caso de Estudio

4.1 Descripción del problema

La generación manual de consultas SQL representa una barrera para usuarios no técnicos [8]. Esta problemática se agudiza en escenarios multitermino [16].

4.2 Descripción de la solución

SQL-Agents aborda el problema mediante un pipeline multi-agente orquestado. La Figura 1 muestra el flujo principal.

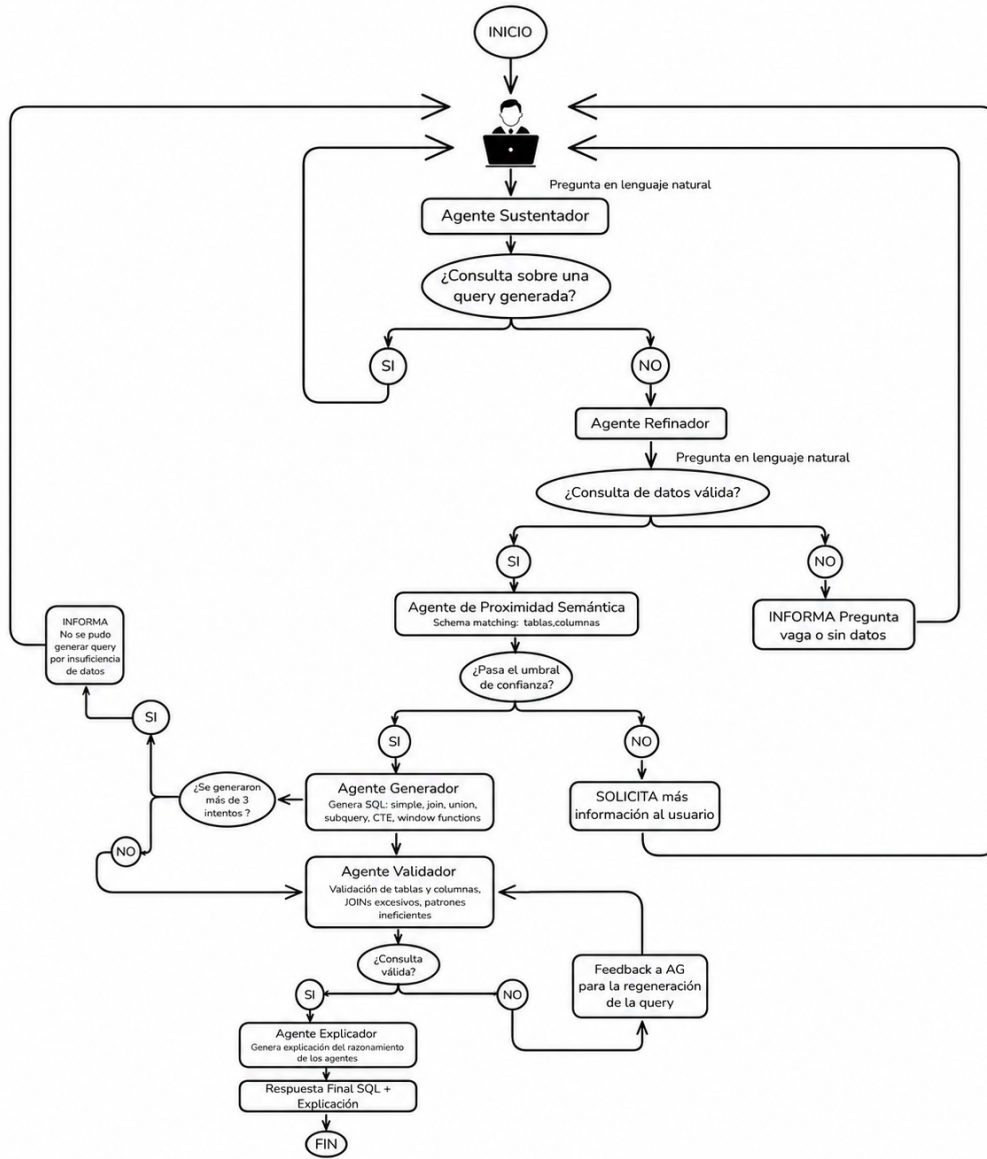


Figura 1: Diagrama de flujo del sistema SQL-Agents.

El flujo general opera así: un **Clasificador de Intención** determina si la entrada es consulta de datos, sustentación o conversacional. Si es consulta, el Orquestador verifica memorias; de no encontrar coincidencia, activa el pipeline: AR, APS, AG, AV y AE. Si el usuario solicita cambio de tema, el sistema reinicia el contexto manteniendo las memorias intactas.

4.3 Arquitectura de agentes

El sistema implementa seis agentes coordinados por un Orquestador sobre LangGraph. La Figura 2 muestra los agentes y sus interacciones; la Figura 3 presenta el módulo CDC y diccionario de datos; y la Figura 4 describe las memorias. La imagen completa se incluye en el Anexo B.

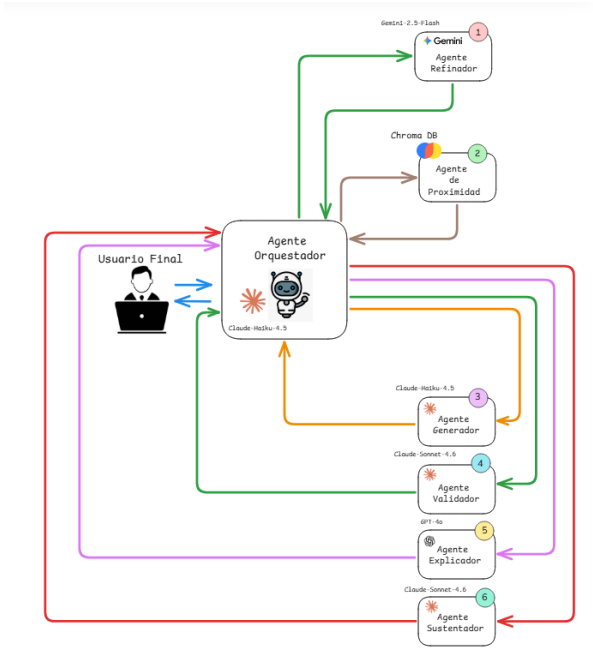


Figura 2: Agentes especializados (AR, APS, AG, AV, AE, AS) y flujo de interacción coordinados por el Orquestador.

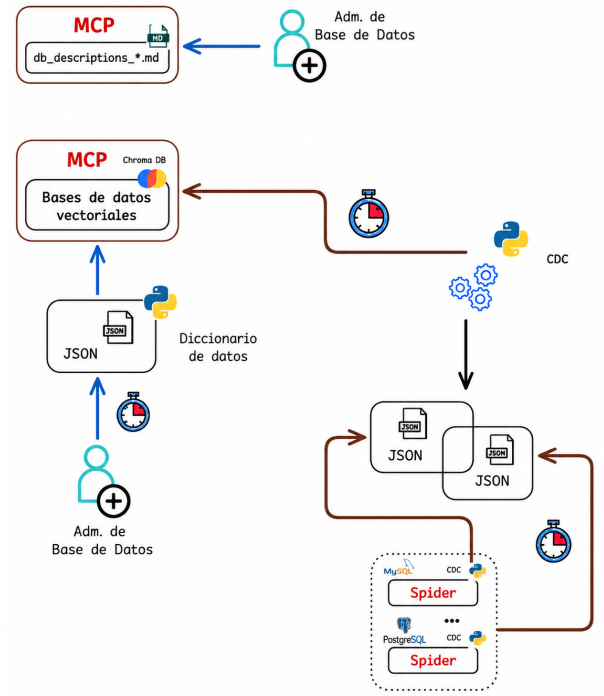


Figura 3: Servidor MCP: schemas DDL, descripciones de dominio y bases vectoriales del esquema. CDC alimenta MySQL/PostgreSQL.

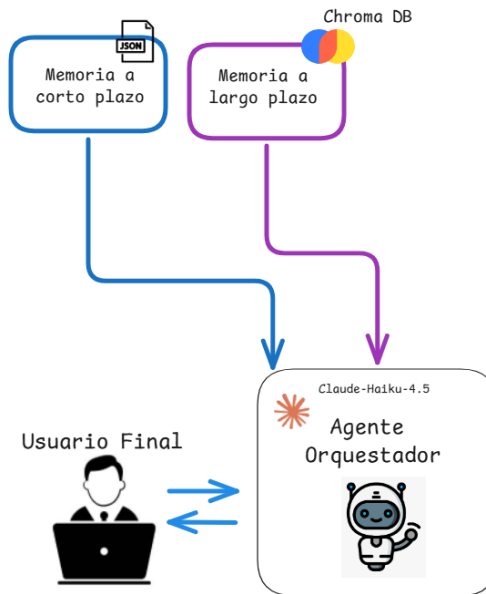


Figura 4: Memoria a corto plazo (RAM de sesión) y memoria a largo plazo (ChromaDB persistente por usuario-base_de_datos-dataset) vinculadas al Orquestador.

El flujo principal sigue: $AR \rightarrow APS \rightarrow AG \rightleftharpoons AV \rightarrow AE$, con AS bajo demanda.

4.3.1 Indexación previa: módulo CDC y diccionario de datos

El CDC extrae el DDL de MySQL y PostgreSQL hacia un JSON. El administrador de la BD provee un **diccionario de datos** con el sig-

nificado semántico de tablas y columnas. El APS indexa ambos en ChromaDB y, en cada consulta, entrega al AG y al AV el subconjunto de tablas y columnas relevantes.

4.3.2 Agente Refinador (AR)

Valida que la pregunta sea una consulta de datos legítima, rechaza DML y conocimiento general, y la refina gramaticalmente. Consume vía MCP la descripción narrativa del dominio activo para no rechazar terminología técnica válida del esquema. Asigna confianza $c_1 \in [0, 1]$.

4.3.3 Agente de Proximidad Semántica (APS)

Realiza búsqueda vectorial en ChromaDB para identificar tablas y columnas relevantes. Sus funciones (`search_tables`, `search_columns`) están publicadas como herramientas MCP, permitiendo su uso tanto desde el pipeline interno como desde clientes externos. Asigna confianza $c_2 \in [0, 1]$ según la similitud media de los candidatos recuperados.

4.3.4 Agente Generador (AG)

Genera SQL (MySQL o PostgreSQL) a partir del esquema provisto por APS. Cuatro skills deterministas cubren: seguridad SQL, corrección de palabras reservadas y búsqueda de JOINS (directos por FK —*Foreign Key*— y por BFS —*Breadth-First Search*— sobre el grafo de FK). Opera en ciclo $AG \Rightarrow AV$; asigna confianza $c_3 \in [0, 1]$.

4.3.5 Agente Validador (AV)

Opera en dos planos: (i) **estructural**: verifica columnas y tipos vs. esquema APS; errores no reparables detienen el pipeline, reparables activan el ciclo $AG \Rightarrow AV$. (ii) **Análisis estático**: detecta cruces de tablas sin condición de JOIN (que generan resultados incorrectos por multiplicar todas las combinaciones posibles), JOINS innecesarios, patrones SQL ineficientes y subconsultas excesivamente anidadas. Asigna confianza $c_4 \in [0, 1]$.

4.3.6 Agente Explicador (AE)

Sintetiza en lenguaje simple las tablas usadas, el SQL generado y la confianza de cada agente. Calcula el **WACS**:

$$\text{WACS} = 0,10 c_1 + 0,20 c_2 + 0,30 c_3 + 0,40 c_4$$

Los pesos reflejan la criticidad creciente de cada etapa: mayor peso a las etapas posteriores implica mayor penalización si el agente más cercano a la respuesta final es el menos confiable. Fueron definidos heurísticamente por diseño y no mediante aprendizaje estadístico.

4.3.7 Agente Sustentador (AS)

Se activa bajo demanda para responder preguntas sobre el proceso (“¿por qué esas tablas?”, “¿qué tan seguro estás?”). Sintetiza los razonamientos de cada agente en lenguaje comprensible.

4.3.8 Componentes Auxiliares

- **Memoria a corto plazo** (RAM de sesión): historial de turnos del diálogo actual; mantiene coherencia conversacional.
- **Memoria a largo plazo** (ChromaDB): persiste consultas y razonamientos previos; el Orquestador la consulta para responder directamente cuando detecta una pregunta similar a una ya resuelta en sesiones anteriores.
- **Enriquecimiento contextual**: resuelve referencias anafóricas (“de ellos”, “los mismos”) recuperando la entidad referenciada del historial antes de invocar el pipeline.
- **Cambio de contexto**: el Orquestador decide mediante LLM si la nueva consulta pertenece a un dominio diferente al activo y, si es así, reinicia el estado conversacional para evitar mezcla de contextos.
- **Tool Registry**: registro central que controla qué *skills* puede invocar cada agente, previniendo accesos no autorizados.

4.4 Metodología (CRISP-DM)

Se adopta CRISP-DM [20] adaptada al desarrollo de un sistema multi-agente NL-to-SQL.

5. Protocolo Experimental

El protocolo experimental cubre **cinco fases** sobre la base de datos Spider 1.0 `concert_singer`, evaluando el sistema tanto a nivel de agente individual (Fases 0–2) como de *pipeline* completo (Fases 3–4).

5.1 Base de datos

Se utiliza `concert_singer` [32], con cuatro tablas: `stadium`, `singer`, `concert` y `singer_in_concert` (21 columnas). Se eligió por ofrecer relaciones $M:N$ no triviales que permiten evaluar *joins* multi-tabla y subconsultas. La misma base se carga en MySQL y PostgreSQL para comparación cruzada.

Datasets por agente. Cada agente se evalúa con 10 casos de prueba:

- **AR**: 5 preguntas válidas + 5 inválidas (DML, opinión, fuera de dominio).

- **APS**: 10 preguntas con tablas y columnas esperadas anotadas.
- **AG**: 10 pares (pregunta NL, SQL canónico esperado).
- **AV**: 5 SQLs correctos + 5 con errores estructurales o semánticos.
- **AE / AS**: 10 pares (SQL generado, explicación esperada).

Dataset del orquestador. El dataset cubre **10 escenarios** que ejercitan cada ruta posible del *pipeline*:

- **Rutas 1–4**: Casos de rechazo — pregunta subjetiva (END), DML (AR rechaza), concepto fuera de esquema (APS rechaza) y pregunta de sustentación (AS directo).
- **Rutas 5–10**: Pipeline completo con los seis agentes, cubriendo agregación, JOIN+GROUP BY, subconsultas, DISTINCT y el bucle de corrección AV → AG.

Cada escenario define `expected_tools` con la lista exacta de agentes y *skills* esperadas por ruta, habilitando `ToolCorrectnessMetric`.

5.2 Modelos LLM evaluados

Se evaluaron seis modelos de tres proveedores comerciales (Tabla 2). Por cada proveedor se incluyó un modelo de menor y otro de mayor capacidad, permitiendo comparar si los resultados se generalizan entre arquitecturas distintas.

Tabla 2: Modelos LLM evaluados y justificación de selección.

Modelo	Proveedor	Justificación
GPT-4o-mini	OpenAI	Bajo costo, alta velocidad
GPT-4o	OpenAI	Alto razonamiento analítico
Claude Haiku 4.5	Anthropic	Eficiencia en tareas estructuradas
Claude Sonnet 4.6	Anthropic	Balance precisión-coste
Gemini 2.5 Flash	Google	Velocidad y soporte multilingüe
Gemini 2.5 Pro	Google	Alta capacidad de contexto

5.3 Modelo de generación de embeddings

Los componentes vectoriales del sistema —búsqueda semántica del APS e indexación de la memoria a largo plazo— emplean `intfloat/multilingual-e5-base` [40, 41]. Se eligió por su cobertura multilingüe (100 idiomas, arquitectura XLM-R), alto rendimiento en el benchmark MTEB [41], operación local sin API de pago

y licencia MIT que permite uso comercial. Este modelo se mantiene fijo en todo el proyecto.

5.4 Temperaturas evaluadas y herramienta de métricas (DeepEval)

Se evalúan cinco valores de temperatura que cubren el espectro completo entre comportamiento determinista y mayor diversidad léxica (Tabla 3).

Tabla 3: Valores de temperatura evaluados y su respaldo bibliográfico.

T	Justificación / Fuente
0,0	Determinístico; estándar en NL-to-SQL [33, 34].
0,1	Casi-determinístico con mínima diversidad léxica [7].
0,3	Baja varianza; mencionado en documentación de proveedores [38, 39].
0,5	Equilibrio reproducibilidad-diversidad [7].
0,7	Valor por defecto en LLaMA 2 [47]; mayor creatividad léxica.

Herramienta de métricas: DeepEval y juez LLM. Las métricas semánticas (F = Faithfulness, G = Groundedness) se calculan con **DeepEval** [25] mediante el criterio **GEval** [26], que instruye a un LLM para puntuar la calidad de la salida del agente comparándola con la respuesta esperada según un criterio definido. El juez es fijo: **GPT-4o** con $T=0,0$ [39] en todas las fases, maximizando la reproducibilidad de la evaluación. Este diseño sigue la práctica establecida de LLM-as-judge [49]: un evaluador de alta capacidad con temperatura nula minimiza la varianza del juez, aislando en mayor medida la varianza del agente evaluado. El mismo juez y los mismos criterios se reutilizan en todas las fases del protocolo para garantizar comparabilidad directa entre agentes.

5.5 Fase 0 — Selección de herramienta y espacio de búsqueda

Objetivos. La Fase 0 persigue dos metas sobre el Agente Refinador (AR): *(i)* comparar cinco herramientas de optimización y observabilidad para seleccionar la que se usará en las fases siguientes, y *(ii)* ejecutar una **grilla exhaustiva** de 6 modelos $\times$ 5 temperaturas = 30 combinaciones para determinar el espacio reducido de fases siguientes.

Función objetivo. La fórmula usada en Fase 0 es:

$$\text{combined}_{AR} = \frac{2F + R + 2G - E - B}{5}$$

F (Faithfulness) y G (Groundedness) reciben coeficiente 2 por ser métricas LLM-as-judge que evalúan el significado semántico; R (ROUGE-L) recibe coeficiente 1 por medir coincidencia léxica superficial. Los términos B (Brier Score) y E (ECE) se restan para penalizar la sobreconfianza. El denominador 5 normaliza el rango a $[0, 1]$.

Se implementó un **módulo de métricas común** que garantiza que todas las herramientas calculen exactamente los mismos valores de F , G , R , B y E . Las cinco herramientas evaluaron las mismas 30 combinaciones de forma exhaustiva con 10 preguntas cada una, utilizando el módulo de métricas común; la búsqueda bayesiana de Optuna se emplea únicamente en las fases de optimización individual (Fases 1 y 3). Las cinco herramientas evaluadas son:

- **Optuna** [27]: optimización bayesiana mediante *Tree-structured Parzen Estimator* (TPE) [48], un algoritmo que aprende qué combinaciones son prometedoras y dirige la búsqueda hacia ellas en lugar de explorar el espacio aleatoriamente.
- **LangSmith** [28]: observabilidad de trazas por llamada LLM.
- **Langfuse** [29]: observabilidad LLM en la nube, auto-hosteable.
- **DSPy** [30]: optimización de prompts vía *BootstrapFewShot*.
- **Promptfoo** [31]: evaluación declarativa YAML.

5.6 Fase 1 — Optimización por agente

Punto de partida. A partir de los resultados de la fase anterior se adoptó **Optuna** [27] con **TPESampler** [48] como herramienta de optimización, y se determinó el espacio de búsqueda: cuatro modelos (GPT-4o, Claude Haiku 4.5, Claude Sonnet 4.6, Gemini 2.5 Flash) y cuatro temperaturas $T \in \{0,0,0,3,0,5,0,7\}$.

Objetivo. Encontrar la combinación óptima de modelo y temperatura para cada agente de forma independiente, usando 10 preguntas de prueba por agente y funciones objetivo adaptadas a cada rol. El APS es la excepción: su espacio se limita a la métrica de distancia HNSW (*Hierarchical Navigable Small World*, el índice vectorial de ChromaDB que determina cómo se calculan las similitudes entre embeddings).

5.6.1 Métricas y función objetivo por agente

Para cada agente se calcula un conjunto de métricas independientes y se combinan en un *combined_score* escalar que dirige la búsqueda. La notación que aparece en las fórmulas es la siguiente: $F = \mathbf{Faithfulness} \in [0, 1]$ (cobertura semántica vía GEval LLM-as-judge); $G = \mathbf{Groundedness} \in [0, 1]$ (la salida se apoya en el contexto, no inventa); $R = \mathbf{ROUGE-L} \in [0, 1]$ (similitud léxica determinista, sin LLM; para SQL se aplica con

normalización ortográfica del SQL); $B = \mathbf{Brier Score} \in [0, 1]$ ($\downarrow$); $E = \mathbf{ECE}$ (*Expected Calibration Error*) $\in [0, 1]$ ($\downarrow$). El valor máximo de cada *combined_score* se obtiene sumando los valores máximos de los términos positivos (cada uno ≤ 1) y asumiendo que los términos negativos B y E alcanzan su mínimo 0.

Implementación de las métricas. Las métricas se calculan así:

- F y G : juez LLM fijo (GPT-4o, $T=0,0$) mediante el framework DeepEval [25] con criterio GEval [26] — sin intervención humana.
- R : ROUGE-L determinista (sin LLM) sobre la subsecuencia común más larga; en el AG se normaliza ortográficamente el SQL antes del cálculo (`rouge_1_sql`).
- B y E : calibración probabilística con `scikit-learn` [24] sobre pares (confianza declarada, *outcome* binario).

Función objetivo por agente:

- **AR, AV:** $(2F + R + 2G - E - B)/5$ (máx. 1.0)
- **AE, AS:** $(2F + 2G + 0,5R - E - B)/4,5$ (máx. 1.0; ROUGE-L con peso 0.5 porque distintas formulaciones lingüísticas son equivalentes)
- **AG:** $F + G + R_{SQL} - B - E$ (máx. 3.0; R_{SQL} sobre SQL canonicalizado)
- **APS:** $(F_1^T + F_1^C)/2$ (precisión-recall en tablas y columnas recuperadas)

La tabla completa de métricas por agente se encuentra en la Tabla 17 del Anexo E.

Agente Refinador (AR). Evalúa la corrección semántica de la reformulación. F y G se calculan con DeepEval GEval sobre el par (*refined_query*, *expected_refined*) y sobre (*user_input*, *refined_query*) respectivamente. ROUGE-L mide la similitud léxica entre la reformulación generada y la versión esperada, capturando coincidencias en el orden relativo de las palabras aunque no sean contiguas. B y E calibran la confianza declarada por el agente; para el AR, el *outcome* binario proviene del ground truth del dataset: *outcome*=1 si la pregunta es una consulta válida de BD, *outcome*=0 si es DML, saludo u opinión. El *confidence_score* del AR estima $P(\text{consulta válida})$; Brier y ECE miden cuán bien calibrada está esa probabilidad respecto al label real. La fórmula asigna coeficiente 2 a F y G y coeficiente 1 a R porque Faithfulness y Groundedness son evaluadas por un LLM-as-judge que comprende el significado de la consulta, mientras que ROUGE-L es una métrica léxica que no distingue reformulaciones semánticamente equivalentes

con vocabulario distinto. En el AR, lo crítico es que la consulta refinada *signifique* lo mismo que la original y esté *anclada* en el input del usuario; la coincidencia léxica exacta es secundaria. La normalización por 5 (máximo del numerador cuando $F=G=R=1$, $B=E=0$) mantiene el rango $[0, 1]$:

$$\text{combined}_{AR} = \frac{2F+R+2G-E-B}{5}$$

El valor máximo es 1,0.

Agente de Proximidad Semántica (APS).

Al no usar un LLM, el único hiperparámetro del APS es la métrica de distancia del índice HNSW (*Hierarchical Navigable Small World*) de ChromaDB. Se evalúan las tres opciones disponibles (*cosine*, 12, *ip*) manteniendo fijo el modelo de embeddings (*multilingual-e5-base*). La función objetivo mide la precisión de recuperación mediante F_1 de tablas (F_1^T) y columnas (F_1^C): $\text{combined}_{APS} = (F_1^T + F_1^C)/2$.

Agente Generador (AG). La salida es una sentencia SQL ejecutable. Antes de calcular las métricas, ambas consultas (generada y esperada) se canonizan con *sqlglot* [42] para eliminar diferencias superficiales de alias, mayúsculas y espacios. Sobre el SQL canonicalizado se aplican tres señales:

- F (GEval): cobertura semántica de la consulta generada respecto al SQL esperado.
- G (GEval): el SQL generado se apoya en las tablas y columnas del esquema sin inventar identificadores fuera de él.
- R_{SQL} : similitud léxica sobre tokens SQL normalizados (ROUGE-L con normalización de identificadores y operadores).

El calibrador Brier/ECE usa $\text{outcome}=1$ cuando $R_{SQL} \geq 0,70$, umbral definido heurísticamente como criterio operativo de equivalencia SQL aceptable, aplicado de forma consistente en todos los experimentos del AG. La función objetivo:

$$\text{combined}_{AG} = F + G + R_{SQL} - B - E$$

El valor máximo es 3,0 cuando $F=G=R_{SQL}=1$ y $B=E=0$. La separación MySQL/PostgreSQL es necesaria porque el dialecto SQL difiere (sintaxis de fechas, comillas, tipos): cada base de datos tiene su propio ganador.

Agente Validador (AV). Al igual que el Agente Refinador, el AV se evalúa sobre la **explicación textual del razonamiento** en lugar de solo la decisión binaria. El AV emite dos campos: **reasoning** (texto que justifica la

decisión) y **decision_word** ("correct" o "incorrect"). El dataset incluye **expected_reasoning** y **expected_decision_word** como referencia de evaluación. Las métricas son:

- **Faithfulness** (F): GEval sobre (**reasoning**, **expected_reasoning**) — ¿cubre el razonamiento del AV los mismos puntos que la explicación esperada?
- **Groundedness** (G): GEval con contexto (*refined_query* + SQL + schema) — ¿se apoya el razonamiento en la información disponible sin inventar?
- **ROUGE-L** (R): `rouge_l_score(decision_word, expected_decision_word)` — dado que **decision_word** adopta solo dos valores posibles ("correct" / "incorrect"), ROUGE-L actúa aquí como indicador de coincidencia exacta del veredicto; la señal principal sobre la correctitud del diagnóstico proviene del *outcome* de B y E .
- B , E sobre (**confidence_score**, **outcome**), donde $\text{outcome} = 1$ si la decisión emitida coincide con la esperada, es decir, si el AV identificó correctamente si el SQL era válido o no.

F y G reciben coeficiente 2 por evaluar el *significado* del razonamiento; ROUGE-L recibe coeficiente 1 porque un veredicto puede expresarse de múltiples formas léxicamente distintas pero igualmente válidas. La normalización por 5 mantiene el rango $[0, 1]$:

$$\text{combined}_{AV} = \frac{2F+R+2G-E-B}{5}$$

El valor máximo es 1,0.

Agente Explicador (AE) y Agente Sustentador (AS). Ambos producen lenguaje natural y comparten la misma estructura de evaluación:

- F (GEval): la explicación/sustentación generada cubre los mismos puntos que la respuesta esperada.
- G (GEval): el texto se ancla en el contexto recibido (pregunta, SQL, tablas) sin fabricar información.
- R (ROUGE-L): baseline léxico determinista.
- B y E : calibración del *confidence_score* que ambos agentes emiten en formato JSON estructurado. El $\text{outcome} = 1$ cuando $G \geq 0,80$. Dado que G también integra la función objetivo, existe una dependencia entre la métrica de calidad y el criterio de calibración; se acepta porque el objetivo primario de AE y AS es minimizar alucinaciones y G es el indicador más directo de ese

fenómeno. Se eligió **Groundedness** como criterio de calibración —en lugar de Faithfulness o ROUGE-L— porque: (i) detecta alucinaciones directamente (el agente explica con información no presente en el contexto); (ii) es robusto ante paráfrasis y sinónimos que ROUGE-L penalizaría incorrectamente; y (iii) no exige coincidencia con una referencia fija, lo cual es apropiado en agentes explicativos donde múltiples formulaciones son igualmente válidas.

La fórmula es idéntica para AE y AS: $(2F + 2G + 0,5R - E - B)/4,5$, valor máximo 1,0.

Para AE y AS, F y G reciben coeficiente 2 porque la calidad de una explicación o sustentación se mide principalmente por:

- **Faithfulness** (F): ¿cubre la explicación los mismos puntos que la referencia? Una respuesta que omite información clave es semánticamente incorrecta aunque léxicamente similar.
- **Groundedness** (G): ¿se ancla el texto en el contexto recibido (SQL, tablas, pregunta) sin fabricar datos o razonamientos? Es la métrica que previene las alucinaciones del agente.

ROUGE-L recibe coeficiente 0,5 porque en tareas de generación de lenguaje natural, múltiples formulaciones son semánticamente equivalentes. Dos explicaciones con vocabulario diferente pero que cubren los mismos conceptos son igualmente válidas; un coeficiente alto en ROUGE-L penalizaría ese parafraseo sin justificación técnica. El denominador 4,5 es el máximo del numerador cuando $F=G=R=1$, garantizando que el valor máximo sea exactamente 1,0.

5.7 Fase 2 — Robustez a paráfrasis

Objetivo. Medir la invariancia de cada agente ante reformulaciones lingüísticas, fijando las configuraciones ganadoras de la fase anterior sin aplicar nueva optimización.

Protocolo. Los inputs del dataset de cada agente se transforman en 3 paráfrasis con perturbaciones controladas: (i) *informal_directa* (omisión de determinantes, contracciones), (ii) *reformulada_abierta* (reescritura conversacional), (iii) *vocabulario_alternativo* (sinónimos). Las 3 paráfrasis de una misma consulta comparten la respuesta esperada. Las 10 preguntas originales se reformulan en 3 variantes cada una, generando **30 casos de prueba por agente** (10 preguntas $\times$ 3 variantes) que sustituyen al dataset original en esta fase; no se evalúan adicionalmente las preguntas originales. La evaluación individual abarca cinco agentes: AR, AG, AV, AE y AS. El APS opera durante el pipeline con la configuración ga-

nadora de la fase anterior, sin someterse a una evaluación individual adicional en esta fase.

Métricas. Las mismas de la fase anterior más la desviación estándar σ entre las 3 variantes de cada pregunta, que mide cuánto varía la calidad de respuesta del agente ante distintas formulaciones de una misma consulta.

5.8 Fase 3 — Optimización del orquestador

Objetivo. Medir el desempeño del *pipeline* completo y determinar el mejor modelo y temperatura para el **agente orquestador**, cuyo rol —clasificar la intención del usuario, detectar cambios de contexto y rutear al agente correcto— solo es evaluable cuando todo el pipeline está activo. Los demás agentes mantienen fijas sus configuraciones ganadoras de Fase 1. La búsqueda se ejecuta de forma independiente para MySQL y PostgreSQL.

Dataset. Los 10 escenarios definidos en la Sección 5.1 cubren todas las rutas posibles del pipeline. **ToolCorrectnessMetric** verifica que el pipeline activó los agentes y *skills* correctos por ruta: APS (búsqueda de tablas, columnas y valores), AG (validación de seguridad, cálculo de JOINS) y AV (sintaxis, patrones semánticos, plan de ejecución).

Métricas globales. Se emplean F , G , B y E definidas en la Sección 5.6.1, más C (**ToolCorrectnessMetric** [25]), que verifica que el pipeline activó los agentes y *skills* correctos por ruta. Brier y ECE se calculan contra el *outcome* binario ($R_{SQL} \geq 0,80$, umbral más estricto que el del AG por evaluar el pipeline completo).

El WACS se calcula en dos modalidades según la ruta activada:

- **Caso ideal** (pipeline completo, Rutas 5–10): los cuatro agentes $A=\{\text{AR, APS, AG, AV}\}$ están activos y el WACS se calcula con pesos fijos: AR 0.10, APS 0.20, AG 0.30, AV 0.40.
- **Caso parcial** (Rutas 1–4, rechazo o sustentación): solo un subconjunto $A_{act} \subset A$ se activa. El WACS se normaliza por los pesos de los agentes efectivamente activos para no diluir la confianza:

$$\text{WACS}_{norm} = \frac{\sum_{i \in A_{act}} w_i c_i}{\sum_{i \in A_{act}} w_i}$$

Búsqueda de hiperparámetros del orquestador. Optuna TPE explora el espacio (modelo, temperatura) del **agente orquestador** sobre los 10 escenarios, con la función objetivo

combined_{orch} como señal de calidad. El espacio de búsqueda comprende cuatro modelos y cuatro valores de temperatura seleccionados a partir del análisis de los resultados de la fase inicial. Se ejecutan 10 evaluaciones en total: las 3 primeras son aleatorias para explorar el espacio y las 7 restantes son dirigidas por el algoritmo TPE (*Tree-structured Parzen Estimator*).

La función objetivo es:

$$\text{combined}_{\text{orch}} = F + G + C - B - E \quad (\text{máx.} \approx 3,0)$$

donde F y G son GEval sobre la explicación final del AE, C es ToolCorrectnessMetric, y B/E son Brier/ECE globales sobre los 10 escenarios.

5.9 Fase 4 — Robustez del orquestador

Objetivo. Validar que el *pipeline* completo sea robusto ante variaciones lingüísticas del usuario, fijando los ganadores de Fase 3.

Protocolo. Cada uno de los 10 escenarios de Fase 3 se reformula en 3 variantes (*informal, reformulada, vocabulario alternativo*), generando $10 \times 3 = 30$ entradas por base de datos. El SQL esperado, la explicación de referencia y las herramientas esperadas permanecen fijos; solo varía la redacción de la pregunta del usuario. La función objetivo es la misma de Fase 3: combined_{orch} = $F + G + C - B - E$. La invariancia completa constituye el comportamiento ideal: activar los mismos agentes y producir SQL equivalente independientemente de la redacción del usuario.

6. Resultados

6.1 Fase 0

La grilla exhaustiva (6 modelos $\times$ 5 temperaturas) se ejecutó con todas las herramientas sobre las mismas 30 configuraciones; Optuna, LangSmith, Langfuse y Promptfoo producen métricas idénticas para el mismo modelo/temperatura, validando la consistencia del módulo de métricas común. DSPy genera métricas diferentes porque no evalúa el modelo tal cual, sino que modifica su prompt mediante *BootstrapFewShot*, alterando su comportamiento y reduciendo el combined_score. Gemini 2.5 Flash ($T=0,5$, $S=0,962$) lidera la grilla; GPT-4o-mini y Gemini 2.5 Pro quedan por debajo de sus pares del mismo proveedor. Se seleccionó **Optuna** por ser la única con búsqueda bayesiana automática, visualizaciones reproducibles y ejecución local. La Tabla 4 muestra las métricas por herramienta y la Tabla 5 sus capacidades funcionales.

Tabla 4: Métricas AR por herramienta — ganador Fase 0 (gemini-2.5-flash, $T=0,5$). DSPy[†] aplica *BootstrapFewShot*.

Herramienta	F	G	R	B	E
Optuna	0.959	0.990	0.970	0.004	0.053
LangSmith	0.959	0.990	0.970	0.004	0.053
Langfuse	0.959	0.990	0.970	0.004	0.053
Promptfoo	0.959	0.990	0.970	0.004	0.053
DSPy [†]	0.808	0.886	0.815	0.071	0.068

[†]La optimización del prompt no mejora el rendimiento ($S=0,812$ vs. $S=0,962$ base, $\Delta=-0,150$).

Tabla 5: Capacidades de las herramientas evaluadas.

Herram.	Propósito	Visual.	Opt. HP
Optuna	Búsqueda bayesiana	HTML local	Sí
LangSmith	Trazabilidad LLMs	Web nube	No
Langfuse	Observab. LLMs	Web nube	No
DSPy	Compilación prompt	—	No
Promptfoo	Evaluación YAML	Tabla HTML	No

Opt. HP: indica si la herramienta realiza optimización automática de hiperparámetros (modelo, temperatura).

6.2 Fase 1

La búsqueda bayesiana se muestra eficiente desde los primeros pasos: con solo 10 *trials*, Optuna identifica el mismo ganador que la grilla exhaustiva de la fase anterior — gemini-2.5-flash con $T=0,5$, $S=0,962$ —, lo que sugiere que el espacio modelo $\times$ temperatura es explorable sin necesidad de evaluaciones exhaustivas (Figura 5).

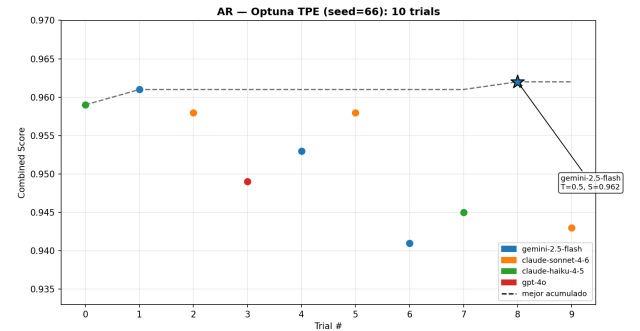


Figura 5: AR — Optuna TPE: combined_score por trial (0–9). La estrella (★) indica el ganador: gemini-2.5-flash $T=0,5$, $S=0,962$.

El APS presenta un comportamiento particular: las tres métricas de similitud HNSW (cosine, l2, ip) producen idéntico combined (0,842) con vectores L2-normalizados, lo que sugiere una invarianza práctica del embedding multilingual-e5-base en este conjunto de evaluación. Se adopta cosine por convención estándar (Tabla 6).

Tabla 6: Trials APS — similitud HNSW con multilingual-e5-base.

Similitud	F_1^T	F_1^C	Combined
cosine (ganador)	0.927	0.757	0.842
12	0.927	0.757	0.842
ip	0.927	0.757	0.842

El hallazgo más relevante de Fase 1 es que no existe un único modelo ganador para todos los roles: Gemini obtuvo el mejor desempeño en el AR —cuya tarea exige reformulación semántica— mientras que Claude lideró en AG y AV, donde predomina la precisión estructural del SQL (Tabla 7). Esta divergencia de ganadores aporta evidencia preliminar compatible con la hipótesis de especialización por rol. En cuanto al APS, la recuperación de tablas es alta ($F_1^T=0,927$), pero la de columnas es menor ($F_1^C=0,757$), sugiriendo margen de mejora en la identificación de atributos específicos.

Tabla 7: Ganadores Fase 1 por agente.

Agente	Modelo	T	Combined
AR	gemini-2.5-flash	0.5	0.962
APS	cosine / e5-base	—	0.842
AG-MySQL [†]	claude-haiku-4-5	0.3	2.700
AG-Postgres [†]	claude-haiku-4-5	0.3	2.687
AV-MySQL [†]	claude-sonnet-4-6	0.7	0.956
AV-Postgres [†]	claude-haiku-4-5	0.0	0.955
AE	gpt-4o	0.3	0.792
AS	claude-sonnet-4-6	0.7	0.870

Combined: función objetivo optimizada por Optuna. Rango [0, 1] para AR, APS, AV, AE, AS; rango [0, 3] para AG (incluye R_{SQL}). [†]AG y AV se evaluaron independientemente para MySQL y PostgreSQL por diferir el dialecto SQL.

6.3 Fase 2

La tabla siguiente muestra la variación del *combined_score* (función objetivo de cada agente) al pasar de las preguntas originales a las 3 paráfrasis. Valores negativos indican caída de rendimiento ante reformulaciones.

Tabla 8: Variación de la función objetivo ante paráfrasis. Δ = resultado_con_paráfrasis − resultado_original.

Agente	Δ MySQL	Δ Postgres
AR	−0,060	−0,060
AG	−0,045	−0,046
AV	−0,010	−0,089
AE	+0,013	+0,013
AS	−0,052	−0,052

APS no fue reoptimizado en Fase 2; se mantuvo la configuración ganadora de Fase 1 (métrica cosine, modelo multilingual-e5-base).

AV en MySQL presenta la menor variación de la función objetivo ($\Delta=-0,010$): su clasificación binaria apenas varía ante reformulaciones. **AE** mejora ligeramente ($\Delta=+0,013$), lo que sugiere que las paráfrasis no perjudican la generación de explicaciones. El agente más afectado es **AV** en PostgreSQL ($\Delta=-0,089$, −9,3 %), seguido de **AR** ($\Delta=-0,060$, −6,2 %) y **AS** ($\Delta=-0,052$, −6,0 %). **AG** en PostgreSQL ($\Delta=-0,046$) también presenta caída, posiblemente atribuible a que las paráfrasis informales dificultan el mapeo de términos al esquema. Los resultados completos por métrica se encuentran en los Anexos.

6.4 Fase 3

La búsqueda Optuna identificó **claude-haiku-4-5** ($T=0,3$) como configuración ganadora del orquestador tanto para MySQL como para PostgreSQL. MySQL obtiene un *combined* de 2.744 y PostgreSQL de 2.554 (sobre un máximo teórico de 3.0). La métrica *C* (ToolCorrectnessMetric) es 0.991 en MySQL y 0.800 en PostgreSQL, lo que refleja un desempeño inferior en PostgreSQL dentro del conjunto de escenarios evaluado.

Tabla 9: Configuración ganadora del orquestador por base de datos. Combined = $F+G+C-B-E$, máx. $\approx 3,0$.

Métrica	MySQL	PostgreSQL
Modelo	claude-haiku-4-5	claude-haiku-4-5
Temperatura	0.3	0.3
F	0.865	0.858
G	0.964	0.967
C	0.991	0.800
R_{SQL}	0.591	0.575
B	0.005	0.005
E	0.071	0.066
Combined	2.744	2.554

R_{SQL} se reporta como referencia informativa; no integra la función objetivo. Se usa como criterio de *outcome* para B y E : outcome=1 si $R_{SQL} \geq 0,80$.

6.5 Fase 4

La Tabla 10 compara los resultados de Fase 3 (10 escenarios originales) con Fase 4 (30 paráfrasis, 3 por escenario). En MySQL el *combined* cae de 2.744 a 2.488 ($\Delta=-9,3$ %), principalmente por la reducción en C (−0,100) y el incremento de B (+0,092). En PostgreSQL la caída es notablemente menor: de 2.554 a 2.495 ($\Delta=-2,3$ %), con C incluso mejorando (+0,100); esta mejora puntual probablemente responde al reducido tamaño del conjunto de evaluación y no debe interpretarse como una mejora sistemática. El sistema alcanza un índice de confianza normalizado de **82.9 %** en

MySQL y **83.2 %** en PostgreSQL, lo que sugiere robustez semántica ante variaciones lingüísticas en este conjunto de evaluación.

Tabla 10: Orquestador: Fase 3 (10 escenarios) vs. Fase 4 (30 paráfrasis). Ganador: claude-haiku-4-5, $T=0,3$. $B, E \downarrow$; Combined = $F+G+C-B-E$, máx. $\approx 3,0$.

	Base de datos	F	G	C	B	E	Combined
Fase 3	MySQL	0.865	0.964	0.991	0.005	0.071	2.744
	Postgres	0.858	0.967	0.800	0.005	0.066	2.554
Fase 4	MySQL	0.822	0.907	0.891	0.097	0.034	2.488
	Postgres	0.786	0.936	0.900	0.091	0.036	2.495
$\Delta (F4-F3)$	MySQL	-0,043	-0,057	-0,100	+0,092	-0,037	-0,256 (-9,3 %)
	Postgres	-0,072	-0,031	+0,100	+0,086	-0,030	-0,059 (-2,3 %)
Score norm.	MySQL	2,488/3,0 = 82,9 %					
	Postgres	2,495/3,0 = 83,2 %					

Combined = $F+G+C-B-E$, máx. $\approx 3,0$. Los valores pueden presentar diferencias de $\pm 0,001$ por redondeo en decimales intermedios.

7. Despliegue

SQL-Agents adopta una **arquitectura de microservicios**: cada agente especializado corre en su propio contenedor Docker con un servidor FastAPI que expone el *endpoint* `POST /invoke`. El Orquestador actúa como punto de entrada único y llama a cada agente por HTTP, habilitando escalado, sustitución y monitoreo independiente por servicio.

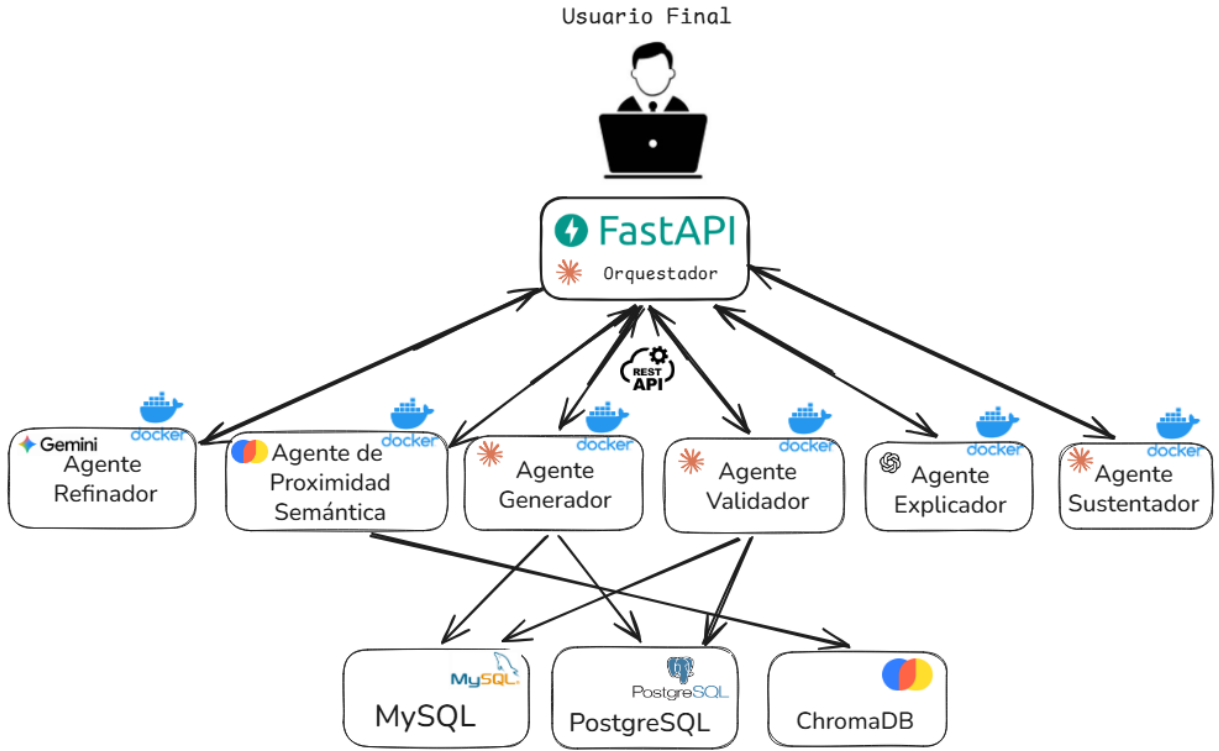


Figura 6: Arquitectura de despliegue: 7 contenedores Docker (orquestador + 6 agentes) comunicados por endpoints REST (`POST /invoke`) a través de una red privada. Modelo ganador por agente: AR gemini-2.5-flash, AG claude-haiku-4-5, AV claude-sonnet-4-6 (MySQL) / claude-haiku-4-5 (Postgres), AE gpt-4o, AS claude-sonnet-4-6, Orquestador claude-haiku-4-5.

El arranque del sistema se gestiona con `docker-compose up --build`. Los contenedores de agentes deben estar *healthy* antes de que el Orquestador arranque, garantizando disponibilidad total del pipeline:

```

docker-compose up --build
ar      :8001 healthy
aps     :8002 healthy
ag      :8003 healthy

```

```

av      :8004 healthy
ae      :8005 healthy
as_agent :8006 healthy
orch    :8000 ready

```

Cada agente expone un *endpoint* REST (`POST /invoke`) que el Orquestador invoca por HTTP. Los contenedores se comunican a través de una red privada Docker: el orquestador resuelve cada agente por nombre de servicio, habilitando esca-

lado, sustitución y monitoreo independiente por componente.

8. Conclusiones

Los resultados obtenidos con **SQL-Agents** sugieren que la especialización multi-agente con el patrón *Agent Skills* + MCP es viable y prometedora bajo el conjunto experimental evaluado, con evidencia en los siguientes aspectos:

- **Distintos roles pueden beneficiarse de modelos diferentes en este conjunto experimental.** Los experimentos sugieren que no existe un único LLM óptimo para todo el pipeline: modelos de precisión técnica (claude-haiku-4-5) destacan en tareas SQL estructuradas, mientras que modelos de mayor capacidad analítica (claude-sonnet-4-6, gpt-4o) son preferidos en razonamiento y generación de lenguaje natural (ver Tabla 7).
- **La calidad de la evaluación es un factor determinante para interpretar el desempeño del agente.** Los resultados sugieren que la normalización ortográfica del SQL, el umbral correcto para el criterio binario y el uso de ejemplos genéricos en los prompts son factores que inciden en que los resultados reflejen la calidad del sistema y no se vean distorsionados por el propio proceso de medición.
- **Evidencia de robustez ante variaciones lingüísticas en el conjunto evaluado.** Al evaluar con 30 paráfrasis por agente, los agentes especializados en SQL (AG-MySQL, AG-Postgres, AV-MySQL) presentan caídas inferiores al 2% en el *combined_score* (calculado como Δ/base , donde la base es el *combined* de Fase 1). Los agentes de lenguaje natural (AR, AS) y AV-Postgres registran caídas de entre el 6% y el 9,3%, principalmente ante vocabulario alternativo e informal, sin que esto afecte de forma sustancial las métricas semánticas y

de similitud SQL empleadas ni la precisión del enrutamiento del orquestador.

- **Robustez del orquestador (Fase 3 $\rightarrow$ Fase 4, $\Delta < 10\%$).** Con 30 variantes por escenario la puntuación normalizada se mantiene entre 82,9% y 83,2% y $C \geq 0,891$ en Fase 4, lo que sugiere que el orquestador selecciona correctamente los agentes y skills esperados en la mayoría de los escenarios evaluados (detalles en Tabla 10).
- **Agent Skills + MCP es viable arquitectónicamente.** La separación habilidades-LLM/lógica-MCP permitió reutilizar el grafo de JOINS, desacoplar la verificación de esquema del proveedor y sustituir LLMs de forma independiente entre bases de datos. La compatibilidad con múltiples proveedores (OpenAI, Anthropic, Google) se logró sin modificar la interfaz de entrada y salida de cada agente, lo que permite sustituir el LLM de cualquier agente de forma aislada.

9. Trabajos futuros

- **Análisis de sensibilidad del WACS.** Evaluar configuraciones alternativas de pesos para aportar evidencia empírica sobre la robustez del índice.
- **Ampliación de datasets y validación estadística.** Ampliar a más de 10 casos por agente y replicar sobre múltiples esquemas para obtener evidencia estadística generalizable.
- **Agente ejecutor y *Execution Accuracy*.** Añadir un séptimo agente que ejecute el SQL contra la BD viva para medir EX estilo Spider 1.0.
- **Esquemas más complejos y otros motores.** Evaluar sobre BIRD [35] (hasta 30 tablas) y adaptar el AG a motores NoSQL o analíticos (MongoDB, DuckDB, BigQuery).

Anexo B: Arquitectura completa

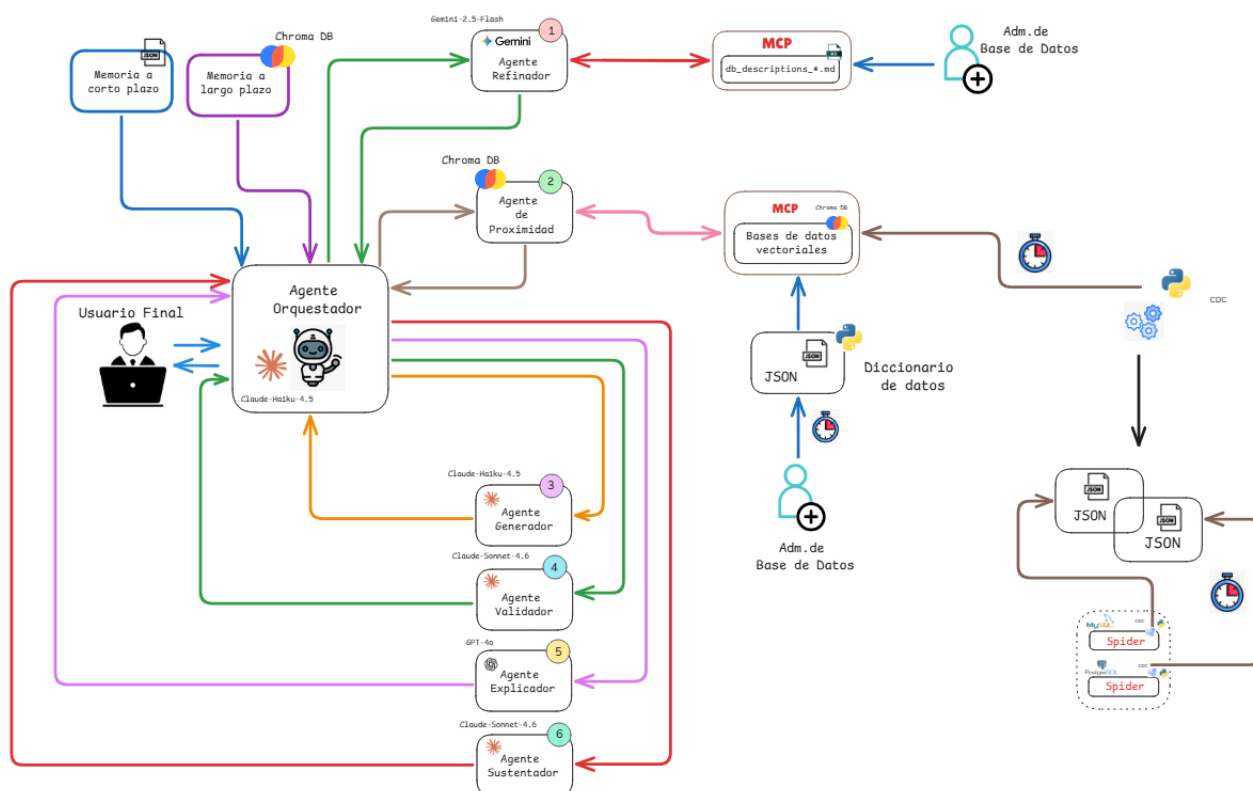


Figura 7: Arquitectura completa del sistema SQL-Agents. Integra los seis agentes con el Orquestador, el servidor MCP (schemas + descripciones + bases vectoriales del esquema), las dos memorias (corto y largo plazo) y el módulo CDC sobre MySQL y PostgreSQL.

Anexo C: TOP-5 trials Fase 1 por agente

Tabla 11: Fase 1 — Top-5 trials por agente (AR, AE, AS). Optuna TPE. *Ganador. †Configuración desplegada en Fases 2–4.

Agente	Modelo	T	F	G	R	B	E	Combined
AR (1)*	gemini-2.5-flash	0.5	0.959	0.990	0.970	0.004	0.053	0.962
AR (2)	gemini-2.5-flash	0.7	0.955	0.985	0.970	0.002	0.041	0.961
AR (3)	claude-haiku-4-5	0.3	0.964	0.971	0.970	0.002	0.041	0.959
AR (4)	claude-sonnet-4-6	0.0	0.964	0.967	0.970	0.002	0.041	0.958
AR (5)	claude-sonnet-4-6	0.5	0.951	0.980	0.970	0.002	0.041	0.958
AE*	gpt-4o	0.3	0.845	0.930	0.461	0.058	0.160	0.792
AE (2)	claude-haiku-4-5	0.7	0.810	0.940	0.338	0.010	0.100	0.791
AE (3)	gpt-4o	0.7	0.840	0.922	0.480	0.138	0.080	0.788
AE (4)	claude-haiku-4-5	0.5	0.764	0.965	0.327	0.010	0.100	0.780
AE (5)	gpt-4o	0.7	0.770	0.939	0.434	0.058	0.160	0.759
AS*	claude-sonnet-4-6	0.7	0.981	0.912	0.408	0.006	0.068	0.870
AS (2)	claude-haiku-4-5	0.7	0.947	0.903	0.400	0.006	0.061	0.852
AS (3)	claude-sonnet-4-6	0.0	0.922	0.899	0.404	0.167	0.130	0.788
AS (4)	gpt-4o	0.7	0.905	0.855	0.315	0.261	0.229	0.708
AS (5)	gpt-4o	0.3	0.909	0.833	0.314	0.261	0.229	0.700

*Ganador seleccionado.

Tabla 12: Fase 1 — AG (NL→SQL): Top-5 trials por base de datos. $R = \text{rouge_1_sql}$. *Ganador. Valor máx. 3.0.

Base de datos	Modelo	T	F	G	R_{SQL}	B	E	Combined
MySQL*	claude-haiku-4-5	0.3	0.822	0.957	0.978	0.003	0.054	2.700
MySQL (2)	claude-haiku-4-5	0.3	0.822	0.949	0.975	0.003	0.054	2.689
MySQL (3)	claude-haiku-4-5	0.7	0.818	0.946	0.975	0.003	0.054	2.682
MySQL (4)	claude-haiku-4-5	0.0	0.803	0.952	0.975	0.003	0.056	2.671
MySQL (5)	claude-sonnet-4-6	0.7	0.780	0.906	0.960	0.018	0.102	2.526
Postgres*	claude-haiku-4-5	0.3	0.814	0.959	0.967	0.003	0.050	2.687
Postgres (2)	claude-haiku-4-5	0.3	0.817	0.950	0.966	0.002	0.046	2.685
Postgres (3)	claude-haiku-4-5	0.0	0.810	0.950	0.967	0.003	0.053	2.671
Postgres (4)	claude-haiku-4-5	0.3	0.811	0.945	0.970	0.003	0.052	2.671
Postgres (5)	claude-sonnet-4-6	0.3	0.773	0.929	0.951	0.010	0.092	2.551

Optuna TPE, 10 trials. $R_{SQL} = \text{rouge_1_sql}$ registrado por el experimento. MySQL: ganador claude-haiku-4-5 $T=0,3$ ($S=2,700$); Postgres: ganador claude-haiku-4-5 $T=0,3$ ($S=2,687$).

Tabla 13: Fase 1 — AV (Validador): Top-5 trials por base de datos. *Ganador. R =ROUGE-L sobre decision_word.

Base de datos	Modelo	T	F	G	R	B	E	Combined
MySQL*	claude-sonnet-4-6	0.7	0.980	0.927	1.000	0.001	0.034	0.956
MySQL (2)	claude-sonnet-4-6	0.0	0.966	0.930	1.000	0.001	0.030	0.952
MySQL (3)	claude-sonnet-4-6	0.7	0.966	0.912	1.000	0.001	0.030	0.945
MySQL (4)	gemini-2.5-flash	0.5	0.984	0.871	1.000	0.003	0.049	0.932
MySQL (5)	gemini-2.5-flash	0.7	0.940	0.896	1.000	0.003	0.046	0.925
Postgres*	claude-haiku-4-5	0.0	0.982	0.922	1.000	0.001	0.031	0.955
Postgres (2)	claude-haiku-4-5	0.0	0.977	0.925	1.000	0.001	0.030	0.955
Postgres (3)	gemini-2.5-flash	0.7	0.984	0.915	1.000	0.003	0.054	0.948
Postgres (4)	claude-haiku-4-5	0.3	0.987	0.896	1.000	0.001	0.028	0.947
Postgres (5)	gpt-4o	0.7	0.990	0.892	1.000	0.002	0.042	0.944

MySQL: ganador claude-sonnet-4-6 $T=0,7$ ($S=0,956$); Postgres: ganador claude-haiku-4-5 $T=0,0$ ($S=0,955$).

Anexo D: Resultados detallados Fase 2 (robustez a paráfrasis)

Tabla 14: Métricas evaluadas por agente en Fase 2. $F-E$ idénticas a Fase 1.

Agente	F	G	R	F_1	B	E
AR	✓	✓	✓	—	✓	✓
APS	—	—	—	✓	—	—
AG	✓	✓	R_{SQL}	—	✓	✓
AV	✓	✓	✓	—	✓ [†]	✓
AE	✓	✓	✓	—	✓	✓
AS	✓	✓	✓	—	✓	✓

F_1 : métricas de recuperación (F_1^T tablas, F_1^C columnas) usadas solo por APS. APS no usa Brier/ECE (no genera confianza probabilística).

Tabla 15: Comparación Fase 1 vs Fase 2. $\Delta=F2-F1$. Seed=66, 4 modelos $\times$ 4 temperaturas.

Agente	Modelo	F			G			R			B		E		Δ Comb
		F1	F2	δ	F1	F2	δ	F1	F2	δ	F1	F2	F1	F2	
AR	gemini-2.5-flash, T=0.5	0.959	0.917	-0,042	0.990	0.968	-0,022	0.970	0.796	-0,174	0.004	0.004	0.053	0.051	-0,060
AG-MySQL	claude-haiku-4-5, T=0.3	0.822	0.816	-0,006	0.957	0.922	-0,035	0.978	0.975	-0,003	0.003	0.003	0.054	0.055	-0,045
AG-Postgres	claude-haiku-4-5, T=0.3	0.814	0.813	-0,001	0.959	0.920	-0,039	0.967	0.964	-0,003	0.003	0.003	0.050	0.053	-0,046
AV-MySQL	claude-sonnet-4-6, T=0.7	0.980	0.959	-0,021	0.927	0.924	-0,003	1.000	1.000	± 0	0.001	0.001	0.034	0.033	-0,010
AV-Postgres	claude-haiku-4-5, T=0.0	0.982	0.919	-0,063	0.922	0.856	-0,066	1.000	0.900	-0,100	0.001	0.065	0.031	0.053	-0,089
AE	gpt-4o, T=0.3	0.845	0.847	+0,002	0.930	0.889	-0,041	0.461	0.510	+0,049	0.058	0.037	0.160	0.067	+0,013
AS	claude-sonnet-4-6, T=0.7	0.981	0.881	-0,100	0.912	0.909	-0,003	0.408	0.335	-0,073	0.006	0.031	0.068	0.036	-0,052

AR/AV: $(2F + R + 2G - E - B)/5$. AE/AS: $(2F + 2G + 0,5R - E - B)/4,5$. AG: $F + G + R_{SQL} - B - E$. Fase 2 aplica las paráfrasis con el mismo modelo ganador de Fase 1; Δ Comb muestra la caída por variación lingüística.

Anexo E: Datasets y métricas Fase 1

Tabla 16: Datasets de prueba por agente (10 casos cada uno).

Agente	N	Composición
AR	10	5 consultas válidas (casing, paráfrasis, expansión) + 5 inválidas (DML, opinión, fuera de dominio)
APS	10	Preguntas Spider <code>concert_singer</code> con tablas/columnas esperadas
AG	10	Pares (pregunta, SQL canónico) Spider
AV	10	5 SQL válidos + 5 con errores (sintácticos y semánticos)
AE	10	Explicaciones esperadas para SQL simple, JOINS, CTEs
AS	10	Preguntas de seguimiento sobre razonamiento del pipeline

Tabla 17: Métricas y función objetivo por agente en Fase 1.

Agente	Rol	F	G	R	F_1	B	E	combined_score (valor máx.)
AR	NL→NL	✓	✓	✓	—	✓	✓	$(2F + R + 2G - E - B)/5$ (1.0)
APS	Retriever	—	—	—	✓	—	—	$(F_1^T + F_1^C)/2$ (1.0)
AG	NL→SQL	✓	✓	R_{SQL}	—	✓	✓	$F + G + R_{SQL} - B - E$ (3.0)
AV	Clasif.	✓	✓	✓	—	✓	✓	$(2F + R + 2G - E - B)/5$ (1.0)
AE	SQL→NL	✓	✓	✓	—	✓	✓	$(2F + 2G + 0,5R - E - B)/4,5$ (1.0)
AS	NL (sustenta)	✓	✓	✓	—	✓	✓	$(2F + 2G + 0,5R - E - B)/4,5$ (1.0)

Tabla 18: Configuraciones ganadoras Fase 1 (Optuna TPE, 10 trials).

Agente	Config desplegada	T	Combined	Métricas clave
AR	gemini-2.5-flash	0.5	0.962	$F=0,959, G=0,990, R=0,970$
APS-MySQL	cosine, e5-base	—	0.842	$F_1^T=0,927, F_1^C=0,757$
APS-Postgres	cosine, e5-base	—	0.842	$F_1^T=0,927, F_1^C=0,757$
AG-MySQL	claude-haiku-4-5	0.3	2.700	$G=0,957, F=0,822$
AG-Postgres	claude-haiku-4-5	0.3	2.687	$G=0,959, F=0,814$
AV-MySQL	claude-sonnet-4-6	0.7	0.956	$F=0,980, G=0,927, R=1,000$
AV-Postgres	claude-haiku-4-5	0.0	0.955	$F=0,982, G=0,922, R=1,000$
AE	gpt-4o	0.3	0.792	$F=0,845, G=0,930$
AS	claude-sonnet-4-6	0.7	0.870	$F=0,981, G=0,912$

Anexo F: Top-5 trials Fase 3 — Orquestador

Tabla 19: Fase 3 — Orquestador: Top-5 trials por base de datos. $C = \text{ToolCorrectnessMetric}$. * Ganador. Valor máx. ≈ 3.0 .

Base de datos	Modelo	T	F	G	C	B	E	Combined
MySQL*	claude-haiku-4-5	0.3	0.865	0.964	0.991	0.005	0.071	2.744
MySQL (2)	claude-haiku-4-5	0.3	0.857	0.948	0.991	0.005	0.072	2.719
MySQL (3)	claude-haiku-4-5	0.7	0.855	0.932	0.991	0.005	0.070	2.703
MySQL (4)	claude-sonnet-4-6	0.7	0.845	0.950	0.981	0.005	0.071	2.699
MySQL (5)	claude-sonnet-4-6	0.7	0.845	0.939	0.991	0.005	0.072	2.698
Postgres*	claude-haiku-4-5	0.3	0.858	0.967	0.800	0.005	0.066	2.554
Postgres (2)	claude-haiku-4-5	0.0	0.865	0.958	0.800	0.005	0.069	2.549
Postgres (3)	gpt-4o	0.3	0.860	0.949	0.800	0.005	0.068	2.536
Postgres (4)	gpt-4o	0.7	0.856	0.952	0.800	0.005	0.069	2.534
Postgres (5)	claude-haiku-4-5	0.7	0.856	0.949	0.800	0.005	0.068	2.533

* Ganador seleccionado. Optuna TPE, 10 trials. MySQL: claude-haiku-4-5 $T=0,3$ ($S=2,744$); Postgres: claude-haiku-4-5 $T=0,3$ ($S=2,554$).

Anexo G: Escenarios del Orquestador (Fase 3)

Los 10 escenarios de Fase 3 son idénticos para MySQL y PostgreSQL. En Fase 4 cada escenario se reformula en 3 variantes lingüísticas, manteniendo el mismo SQL esperado y herramientas esperadas.

ID	Tipo		Pregunta	SQL esperado	Explicación
1	rej.	opinión	Who is the best singer in the world?	—	Subjective opinion; cannot be answered with the database.
2	rej.	DML	Delete all concerts from 2010.	—	DML statement (DELETE); system only processes SELECT queries.
3	rej.	APS	What is the average ticket price per concert?	—	Ticket price data not in the schema.
4	act.	AS	Why did the pipeline use a JOIN between concert and stadium?	—	JOIN needed because query required data from both tables sharing <code>stadium_id</code> .
5	pipeline	simple	What is the maximum and average capacity of all stadiums?	<code>SELECT max(capacity), avg(capacity) FROM stadium</code>	MAX and AVG over the Capacity column.
6	pipeline	JOIN	Show stadium names and the highest year of concerts in each stadium.	<code>SELECT T1.Name, MAX(T2.Year) FROM stadium T1 JOIN concert T2 ON T1.Stadium_ID=T2.Stadium_ID GROUP BY T1.Stadium_ID, T1.Name</code>	JOIN + GROUP BY to compute MAX(Year) per stadium.
7	pipeline	sub-query	Show the names of singers who have not performed in any concert.	<code>SELECT name FROM singer WHERE singer_id NOT IN (SELECT singer_id FROM singer_in_concert)</code>	NOT IN subquery over the bridge table.
8	pipeline	DISTINCT	What are all distinct countries where singers above age 20 are from?	<code>SELECT DISTINCT country FROM singer WHERE age >20</code>	DISTINCT + WHERE filter by age.
9	pipeline	GROUP BY	Show the maximum age of singers in each country.	<code>SELECT country, max(age) FROM singer GROUP BY country</code>	GROUP BY country with MAX aggregation.
10	pipeline	HAVING	List the countries with more than one singer in the database.	<code>SELECT country FROM singer GROUP BY country HAVING COUNT(*) >1</code>	HAVING filter; tests AV correction of WHERE+aggregate to HAVING.

Escenarios 1–4: rechazos y rutas especiales. Escenarios 5–10: pipeline completo con SQL de complejidad creciente.

Bibliografia

- [1] Ojuri, S., Han, T. A., Chiong, R., & Di Stefano, A. (2025). Optimizing text-to-SQL conversion techniques through the integration of intelligent agents and large language models. *Information Processing & Management*, 62(5), 104136. <https://doi.org/10.1016/j.ipm.2025.104136>
- [2] Zhao, X., Zhou, X., & Li, G. (2024). Chat2Data: Interactive data analysis. *Proc. VLDB*, 17(12), 4481–4484.
- [3] Wang, Z., Zhang, R., Nie, Z., & Kim, J. (2024). Tool-Assisted Agent on SQL Inspection and Refinement in Real-World Scenarios. *arXiv preprint*, arXiv:2408.16991.
- [4] Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., & Sun, H. (2023). Exploring Chain-of-Thought Style Prompting for Text-to-SQL. In *Proc. EMNLP 2023*, pp. 1524–1543. arXiv:2305.14215.
- [5] Vichev, S., & Marchev, A. (2024). RAGSQL: Context Retrieval Evaluation on Augmenting Text-to-SQL Prompts. In *Proc. IEEE 12th International Conference on Intelligent Systems (IS 2024)*, pp. 1–6.
- [6] Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pp. 74–81.
- [7] Brown, T., et al. (2020). Language models are few-shot learners. In *Advances in NeurIPS*, vol. 33, pp. 1877–1901.
- [8] Katsogiannis-Meimarakis, G., & Koutrika, G. (2023). A survey on deep learning approaches for text-to-SQL. *The VLDB Journal*, 32(4), 905–936.
- [9] Deng, N., Chen, Y., & Zhang, Y. (2022). Recent advances in text-to-SQL. In *Proc. COLING*, pp. 2166–2187.
- [10] Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). John Wiley & Sons.
- [11] Wang, L., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
- [12] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP*, pp. 3982–3992.
- [13] Chroma (2023). *ChromaDB: The AI-native open-source embedding database*. <https://www.trychroma.com/>
- [14] Chase, H. (2023). *LangChain: Building applications with LLMs*. <https://github.com/langchain-ai/langchain>
- [15] Zhang, Z., et al. (2024). A survey on the memory mechanism of large language model based agents. *arXiv preprint*, arXiv:2404.13501.
- [16] Suhr, A., Iyer, S., & Artzi, Y. (2018). Learning to Map Context-Dependent Sentences to Executable Formal Queries. In *Proc. NAACL-HLT 2018*, pp. 2238–2249. arXiv:1804.06868.
- [17] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [18] LangChain AI (2024). *LangGraph: Stateful multi-actor applications with LLMs*. <https://github.com/langchain-ai/langgraph>
- [19] Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Yin, D., & Che, W. (2023). MAC-SQL: Multi-agent collaboration for text-to-SQL. *arXiv preprint*, arXiv:2312.11242.
- [20] Chapman, P., et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- [21] Es, S., et al. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint*, arXiv:2309.15217.
- [22] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.
- [23] Guo, C., et al. (2017). On calibration of modern neural networks. In *Proc. ICML*, pp. 1321–1330.
- [24] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
- [25] Confident AI (2024). *DeepEval: The open-source LLM evaluation framework*. <https://deepeval.com>
- [26] Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2511–2522. <https://arxiv.org/abs/2303.16634>
- [27] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proc. ACM SIGKDD*, pp. 2623–2631.
- [28] LangChain AI (2024). *LangSmith: LLM Observability and Evaluation Platform*. <https://smith.langchain.com>
- [29] Langfuse (2024). *Langfuse: Open-source LLM engineering platform*. <https://langfuse.com>
- [30] Khattab, O., et al. (2024). DSPy: Compiling declarative language model calls into pipelines. In *Proc. ICLR*.
- [31] Promptfoo (2024). *Promptfoo: Test your prompts, agents, and RAGs*. <https://promptfoo.dev>
- [32] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proc. EMNLP*, pp. 3911–3921.
- [33] Pourreza, M., & Hu, Y. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Proc. NeurIPS*. arXiv:2304.11015.
- [34] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, B. (2023). Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint*, arXiv:2308.15363.
- [35] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., ... & Li, Y. (2023). Can LLM already serve as a database interface? A BIG bench for large-scale database grounded text-to-SQLs. In *Proc. NeurIPS*. arXiv:2305.03111.
- [36] Anthropic (2024). *Building effective agents*. Anthropic Engineering Blog. <https://www.anthropic.com/engineering/building-effective-agents>
- [37] Anthropic (2024). *Model Context Protocol: An open standard for connecting AI agents to external systems*. <https://modelcontextprotocol.io/>

- [38] Anthropic (2024). *Claude: AI assistant by Anthropic — Haiku and Sonnet model family*. <https://www.anthropic.com/claude>
- [39] OpenAI (2024). *GPT-4o and GPT-4o-mini: Multimodal flagship models*. <https://openai.com/gpt-4o>
- [40] Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., & Wei, F. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. arXiv:2212.03533. <https://arxiv.org/abs/2212.03533>
- [41] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). *MTEB: Massive Text Embedding Benchmark*. In *Proc. EACL*, pp. 2014–2037. <https://arxiv.org/abs/2210.07316>
- [42] Mao, T. (2023). *SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine*. <https://github.com/tobymao/sqlglot>
- [43] Amazon Web Services (2024). *Amazon Bedrock — RAG Evaluation Metrics: Faithfulness, Context Relevance, Completeness*. <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-eval-llm-results.html>
- [44] Microsoft (2024). *Azure AI Content Safety — Groundedness Detection*. <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/groundedness>
- [45] Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, 1906–1919.
- [46] Google DeepMind (2024). *Gemini 2.5: Multimodal models family — Flash and Pro variants. Generation configuration: temperature, top-p*. <https://ai.google.dev/gemini-api/docs/models/gemini> & <https://ai.google.dev/gemini-api/docs/text-generation#generation-config>
- [47] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., *et al.* (2023). *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288. <https://arxiv.org/abs/2307.09288>
- [48] Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). *Algorithms for Hyper-Parameter Optimization*. In *Advances in Neural Information Processing Systems 24 (NeurIPS 2011)*, pp. 2546–2554.
- [49] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. arXiv:2306.05685. <https://arxiv.org/abs/2306.05685>